

University of Southampton

Faculty of Engineering and Physical Sciences

Electronics and Computer Science

MSc University of Southampton

by

Yufei Tang

October 2023

ELEC6217

Wireless Transceiver Design and Implementation

Assignment 1

1.1 Problem Restatement

Run 'DigitTrans.m' and view the eye diagram of the received data prior to sampling the signal. From this diagram, you should be able to determine the exact initial sampling point Ninit. Try modifying this value and observe the effect that this has on the constellation pattern of the received symbols. What is the optimum value for Ninit? You should explain your answer and include the figures output of your simulation.

1.2 Content of the answer

The optimum value for Ninit is 1.

1.3 Process of solving a problem

Firstly, the eye diagram in the original file is observed when the initial sampling point is 10, and the result is shown in Figure 1.1:

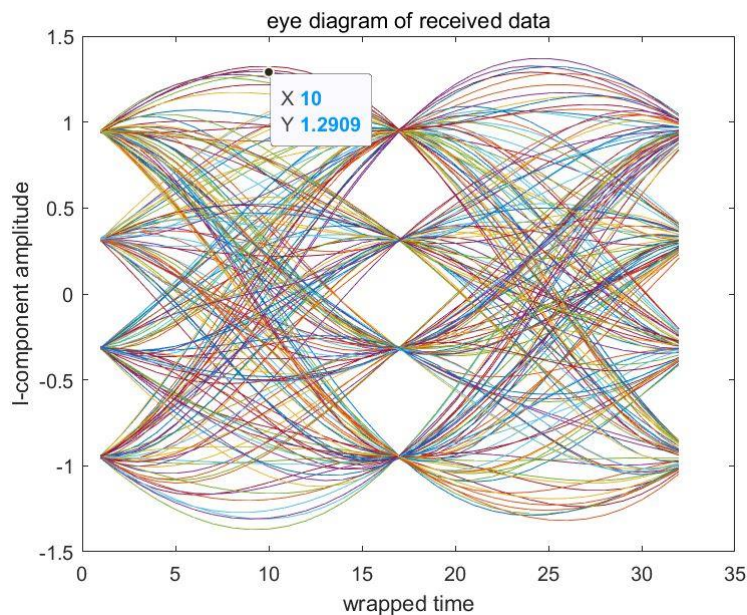


Figure 1.1 Eye diagram with an initial sampling point of 10

Its location can be clearly seen in Figure 1.1. However, it is clear that this is not the optimal sampling point. This is because, as can be seen in Figure 1.1, the sampling point of 10 is not at the point where the "eye" is at its widest opening. And we can easily observe that the optimal sampling point is 17, because in Figure 1.1 the horizontal axis of 17 corresponds to the vertical axis with the largest "eye" opening. However, the code requires that the range of sampling points be 1 to 16, so 17 cannot be chosen as the best sampling point. In fact, there is another pair of open "eyes" in Figure 1.1 that meets the code's sampling point requirement. That is located in the horizontal axis of the initial sampling point 1, its position is half of the "eyes" open maximum. Therefore, sampling point 1 was initially identified as the best initial sampling point. Comparing its eye and constellation diagrams with the original sampling point of 10, it is found that its noise immunity and clarity are obviously of much better quality, while its offset is small, so it is a better sampling point. The following Figures 1.2 and 1.3 represent the constellation diagrams when the initial sampling point is 10 and 1, respectively.

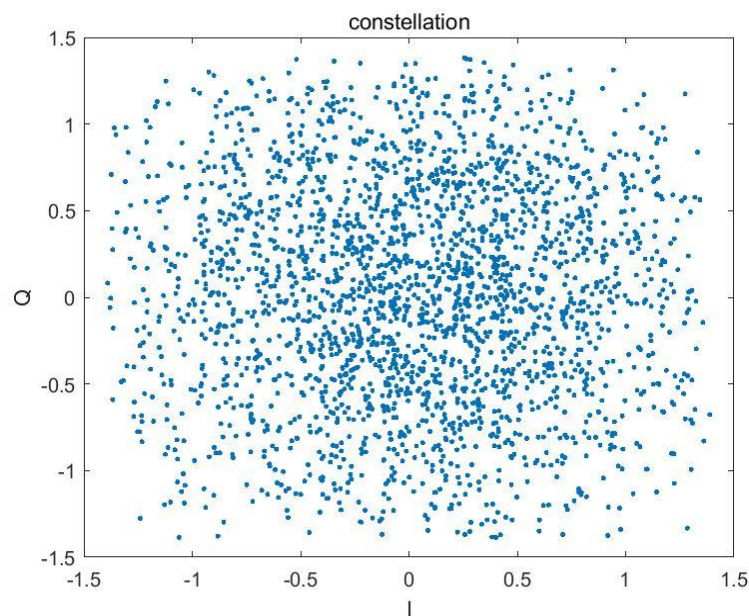


Figure 1.2 Constellation diagram with an initial sampling point of 10

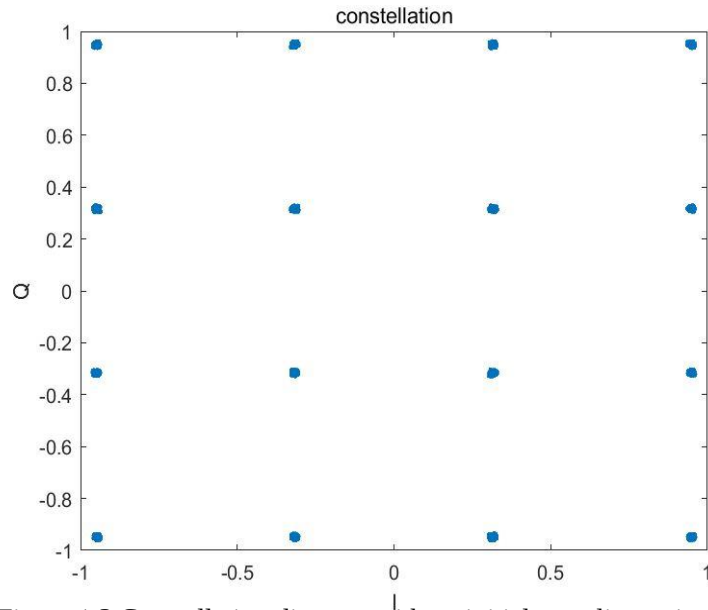


Figure 1.3 Constellation diagram with an initial sampling point of 1

But is that the best sampling point? One should also consider the situation of the constellation diagram of the 16 points closest to the point 17 where the eyes are most open. Figure 1.4 shows the constellation diagram for an initial sampling point of 16.

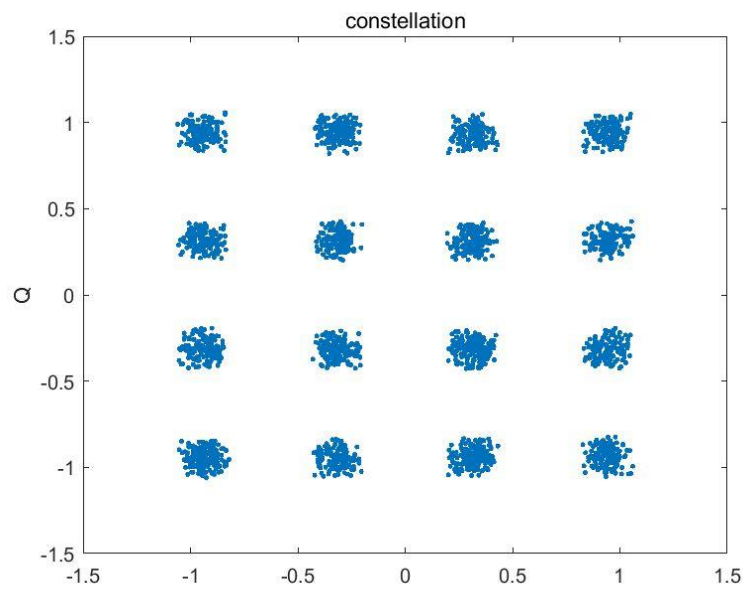


Figure 1.4 Constellation diagram with an initial sampling point of 16

Comparing Figure 1.3, it was finally determined that the best initial sampling point is 1.

1.4 Analysis of results

The size of the "eyes" of the eye diagram reflects the strength of the inter-code crosstalk. The larger the "eyes" are open, and the more upright the eye diagram is, the smaller the inter-code crosstalk is. Conversely, the larger the inter-code crosstalk is. Therefore, from Figure 1.1, we can see that the best sampling point is 17, because this coursework uses oversampling with sampling period 16, so the sampling point 1 one cycle ago is also the

best sampling point.

2.1 Problem Restatement

Modify the channel signal to noise ratio (SNR) to 20 dB, 10 dB and 5 dB, and observe the effect on the eye diagram and the received constellation pattern. Obtain plots of the eye diagram for the different SNR values and write your observations. The reason that the sampling points range from 1 to 16 is because this coursework uses incremental sampling, namely multiple zero values are inserted between two points, which narrows the spectrum and further avoids spectral aliasing. The incremental sampling period of this coursework is 16, so if we start sampling from a part larger than 16, we will lose some symbol information. Therefore, the range of sampling points can only be from 1 to 16.

2.2 Content of the answer

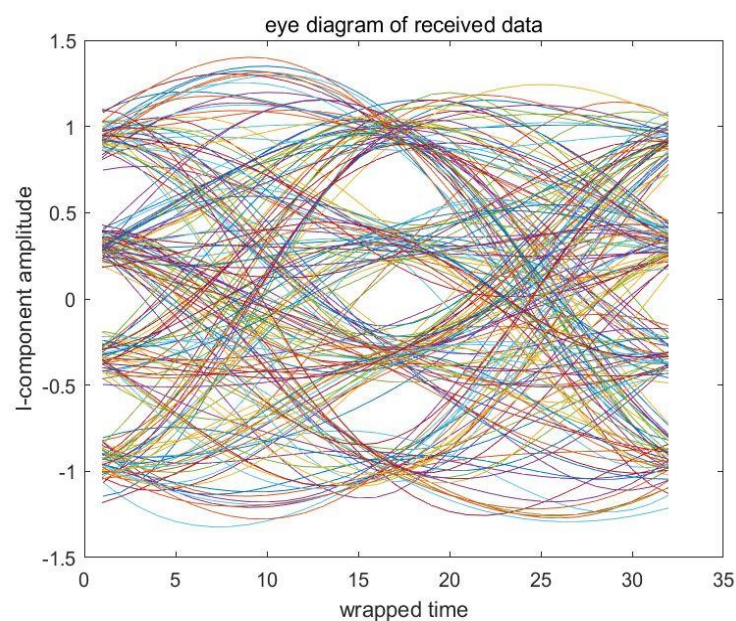


Figure 2.1 Eye diagram with noise = 20dB

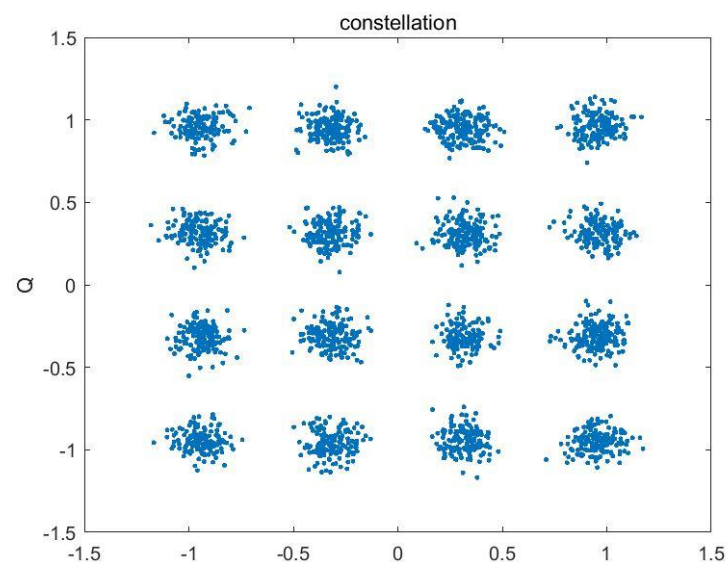


Figure 2.2 Constellation diagram with noise = 20dB

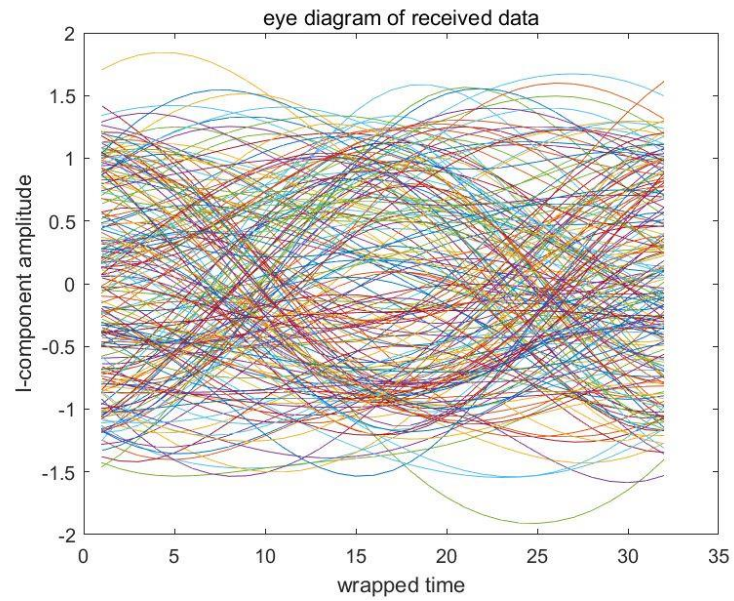


Figure 2.3 Eye diagram with noise = 10dB

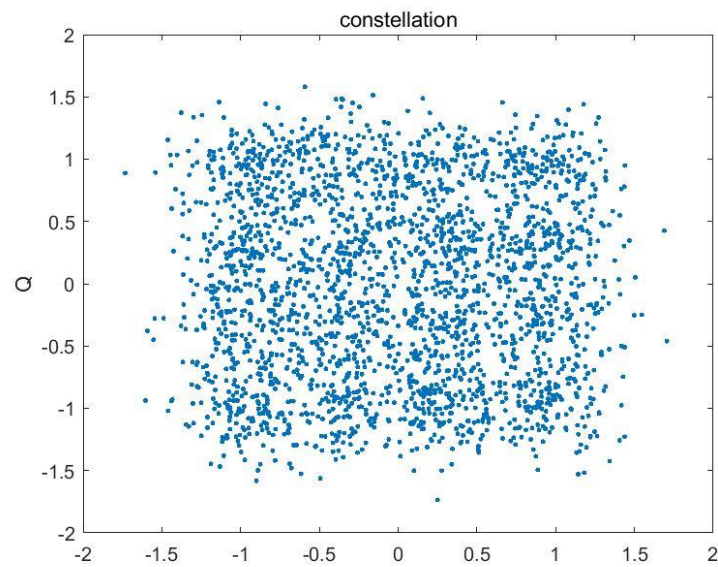


Figure 2.4 Constellation diagram with noise = 10dB

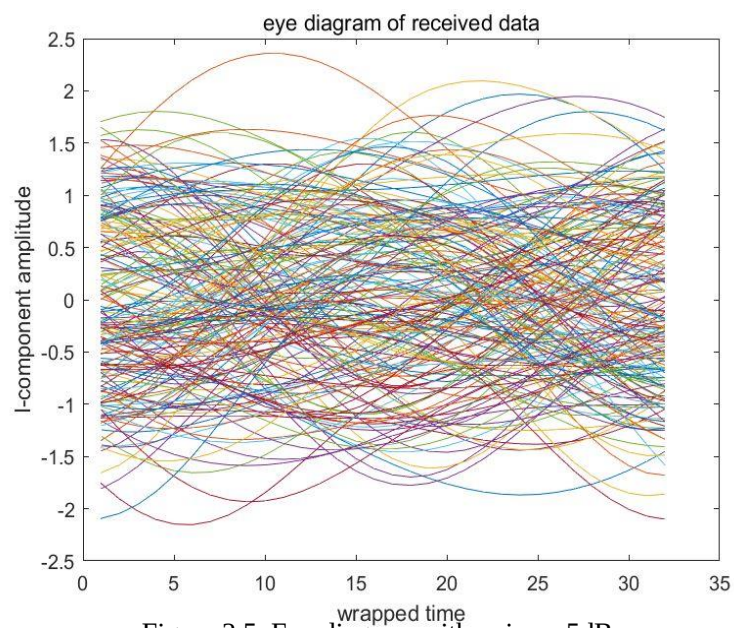


Figure 2.5 Eye diagram with noise = 5dB

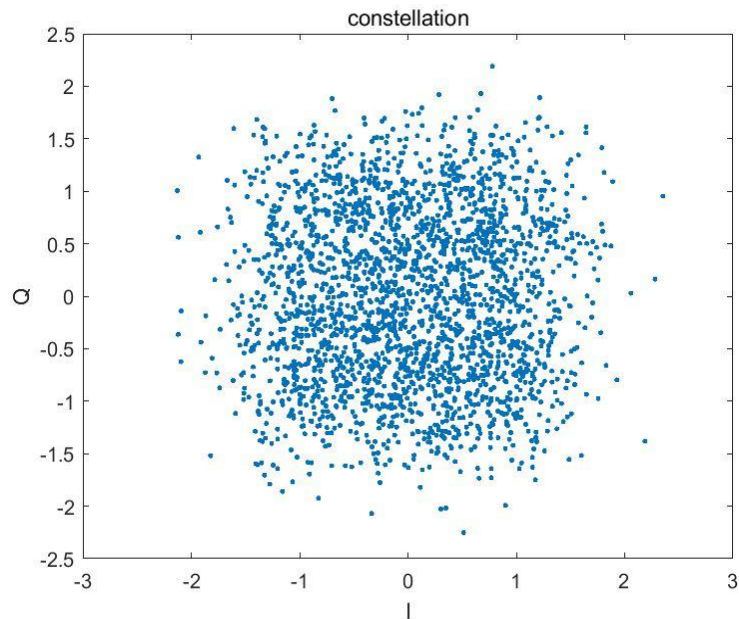


Figure 2.6 Constellation diagram with noise = 5dB

2.3 Process of solving a problem

Since the question requires the observation of the effect of noise on the quality of communication transmission, noise should be established as a single variable. Therefore the default setting for this question is that the initial sampling point is the optimal sampling point, and then the ratio of signal-to-noise is changed to the corresponding eye and constellation diagrams.

2.4 Analysis of results

From Figure 2.1 to Figure 2.6, it can be clearly observed that as the signal-to-noise ratio decreases, the "eyes" in the eye diagram gradually close and the constellation diagram gradually blurs. When the signal-to-noise ratio is less than or equal to 10dB, the points in the constellation diagram are completely indistinguishable. This is because there is too much inter-code crosstalk when the signal-to-noise ratio is too low, leading to indistinguishable portions along the path representing 0 and 1. As for the constellation diagram, too low signal-to-noise ratio causes its constellation points to be unevenly distributed, severely offset from the intended points, the Euclidean distance between neighbouring symbol points increases, and the final diagram presented is very blurred. Therefore, a minimum SNR value of 20 dB should be achieved for data transmission.

3.1 Problem Restatement

Adjust the SNR to 1000 dB (in other words, noise-free) and incorporate a dispersive channel, e.g. by setting $c = [1 \ 0 \ -.7]$ and $c = [1 \ 0 \ -.8]$. Note that you may have to

re-adjust Ninit as you may observe from the eye diagram. Obtain plots of the eye diagram and constellation diagram and comment on them and on the Ninit values.

3.2 Content of the answer

When setting $c = [1 \ 0 \ -0.7]$, needing the value of the Ninit is 13.

When setting $c = [1 \ 0 \ -0.8]$,needing the value of the Ninit is 10.(in order to observe clearly)

See the diagram below for specific details.

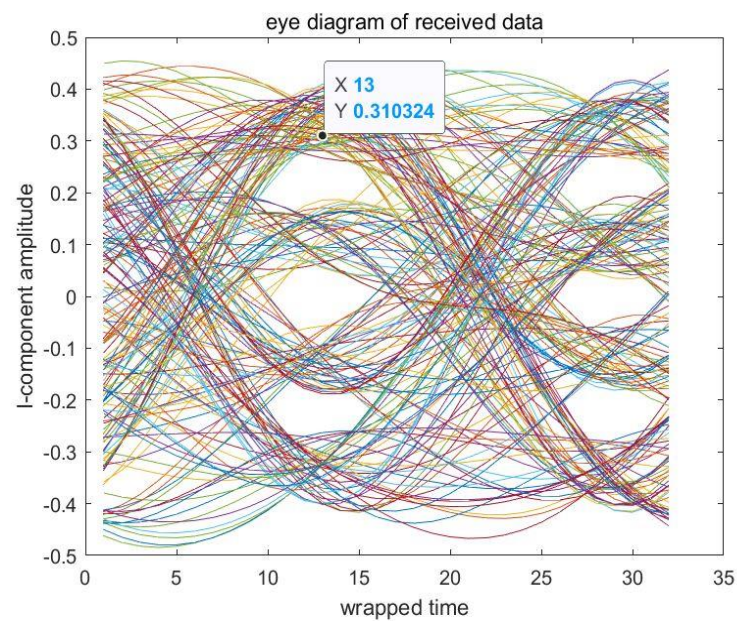


Figure 3.1 Eye diagram with $c = [1 \ 0 \ -0.7]$

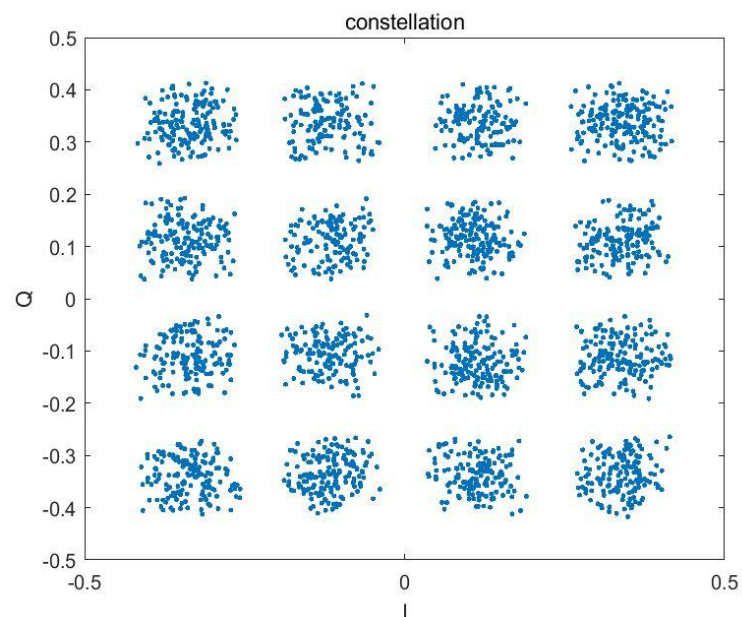


Figure 3.2 Constellation diagram with $c = [1 \ 0 \ -0.7]$ and Ninit = 13

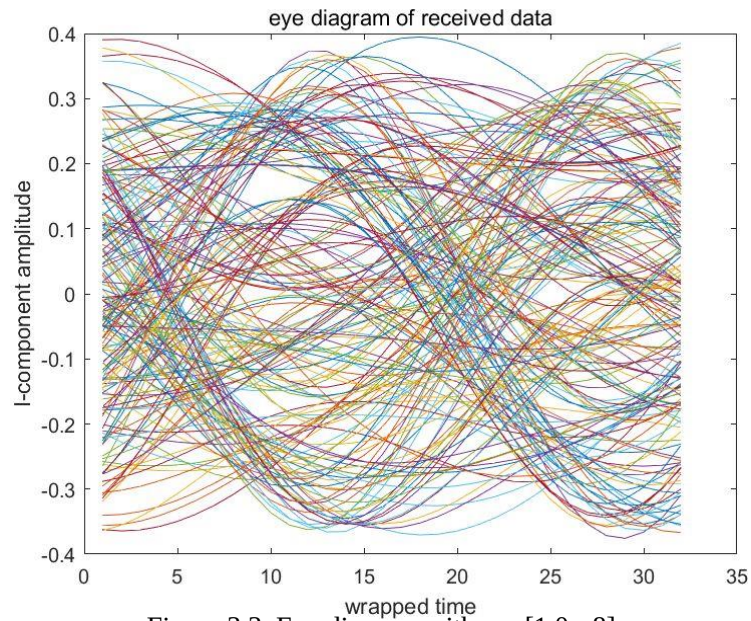


Figure 3.3 Eye diagram with $c = [1 \ 0 \ -.8]$

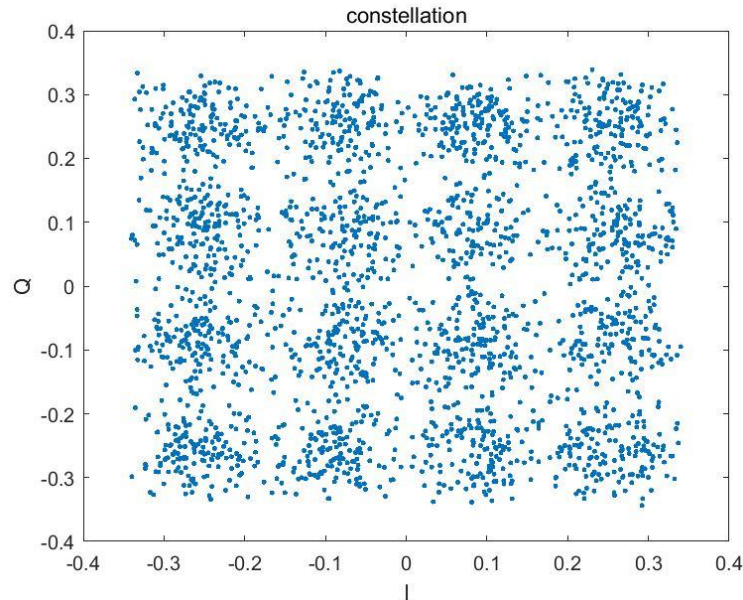


Figure 3.4 Constellation diagram with $c = [1 \ 0 \ -.8]$ and $N_{init} = 10$

3.3 Process of solving a problem

Firstly, N_{init} is kept in its original position and then the signal-to-noise ratio is adjusted back to 1000dB, namely to simulate a noiseless channel, as it is now necessary to observe the effect of the dispersive channel on the communication transmission. The dispersive channel $C=[1 \ 0 \ -.7]$ is set and its corresponding eye and constellation diagrams are shown below.

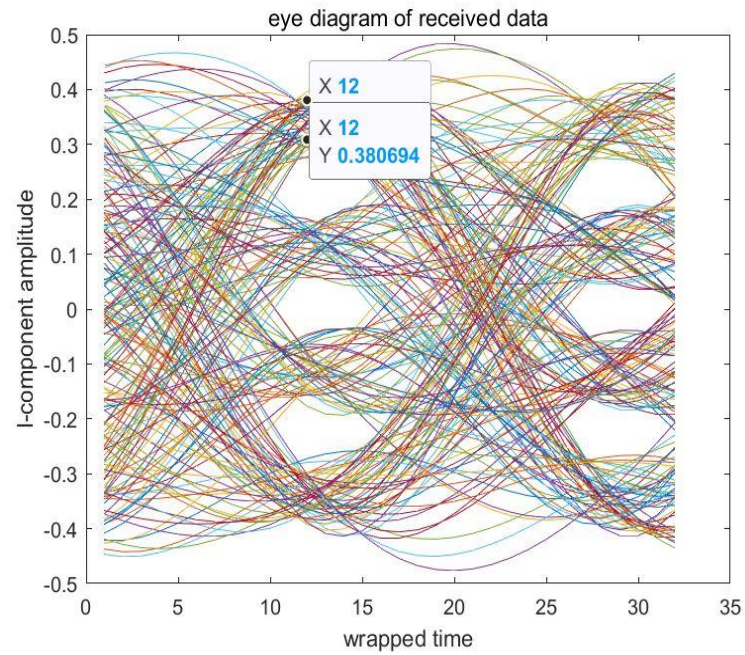


Figure 3.5 Eye diagram with $c = [1 \ 0 \ -.7]$

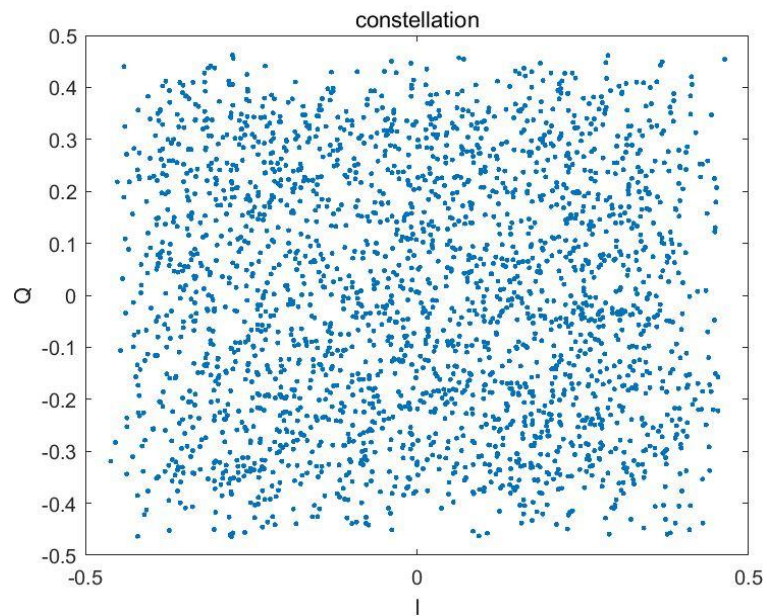


Figure 3.6 Constellation diagram with $c = [1 \ 0 \ -.7]$

From Figure 3.5, it can be found that its optimal sampling point should be 12, so the Ninit value is re-selected. Setting Ninit to 12, its eye and constellation diagrams are obtained as shown below.

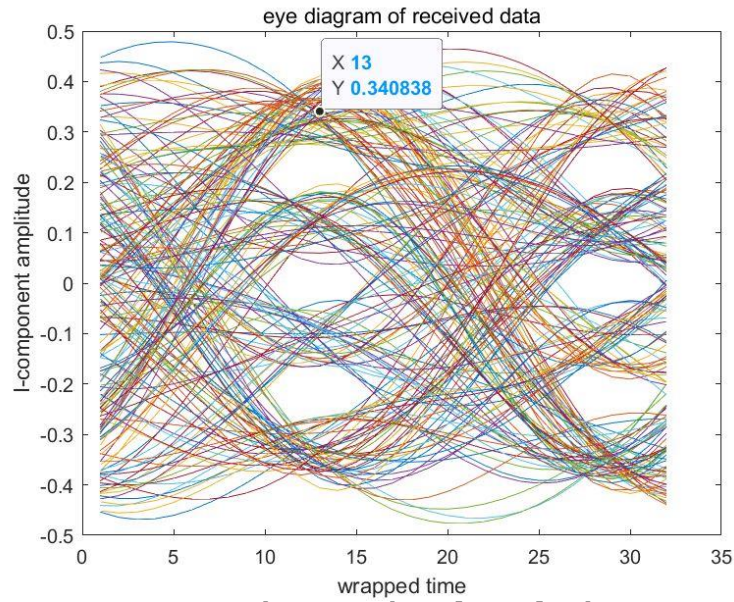


Figure 3.7 Eye diagram with $c = [1 \ 0 \ -0.7]$ and Ninit = 12

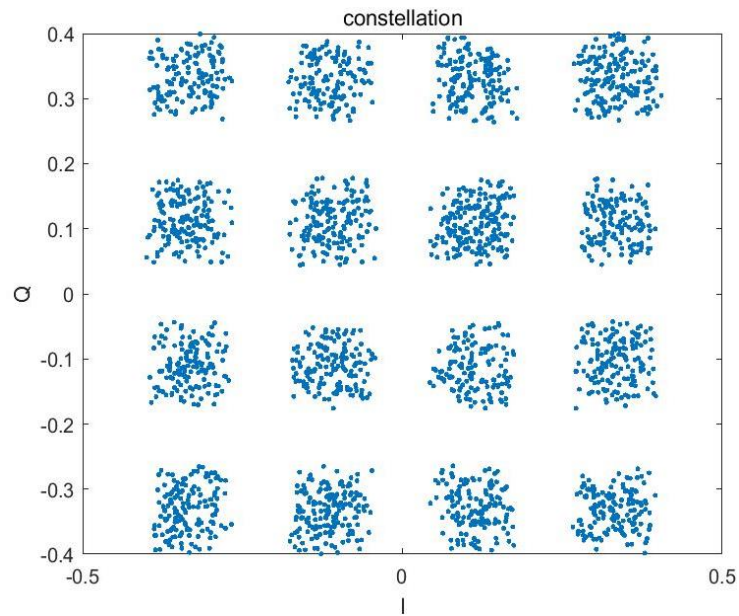


Figure 3.8 Constellation diagram with $c = [1 \ 0 \ -0.7]$ and Ninit = 12

From Figure. 3.7, it can be seen that the "eye" can also be seen at Ninit = 13, so a constellation diagram is drawn for Ninit = 13, namely Figure. 3.2. Comparing Constellation Diagram 3.2 with Constellation Diagram 3.8, it can be clearly seen that Constellation Diagram 3.2 is much clearer, and therefore, it is more suitable for observing dispersive channels' effect on the communication transmission by choosing Ninit = 13. The same approach finds Ninit = 10 as the most suitable initial sampling point for observing the effect of the dispersive channel on the communication transmission when $c = [1 \ 0 \ -0.8]$.

3.4 Analysis of results

A dispersive channel is a channel consisting of components with different propagation speeds or phase delays. In this coursework the dispersive channel is represented by the c vector, and a dispersive channel is simulated when more than one of the three elements in c

is not zero. As can be seen from the above figure, the dispersive channel has a great impact on the transmission process of a communication system. Firstly, it can be seen from the severe distortion of the eye diagram above that it causes huge inter-symbol interference, and from the blurred constellation diagram, it can also be seen that it generates huge BERs due to inter-symbol interference. From the comparison of Figure 3.4 with Figure 3.2, it can be seen that $c = [1 \ 0 \ -.8]$ causes stronger signal distortion than $c = [1 \ 0 \ -.7]$. This is because the third element in the vector c that constitutes the dispersion channel is negative, and the more severe the reverse phase delay of the signal when its generated copy is composed of the first element, the greater the randomness and uncertainty, and the greater the ISI. The observation can be tested by setting the vector c to $c = [1 \ 0 \ .8]$, namely positive, and its corresponding eye and constellation diagrams are shown below:

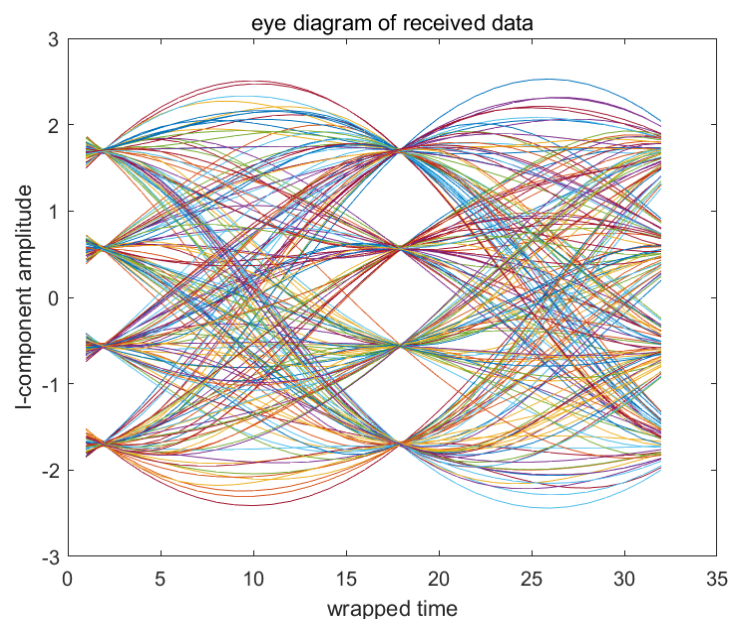


Figure 3.9 Eye diagram with $c = [1 \ 0 \ .8]$

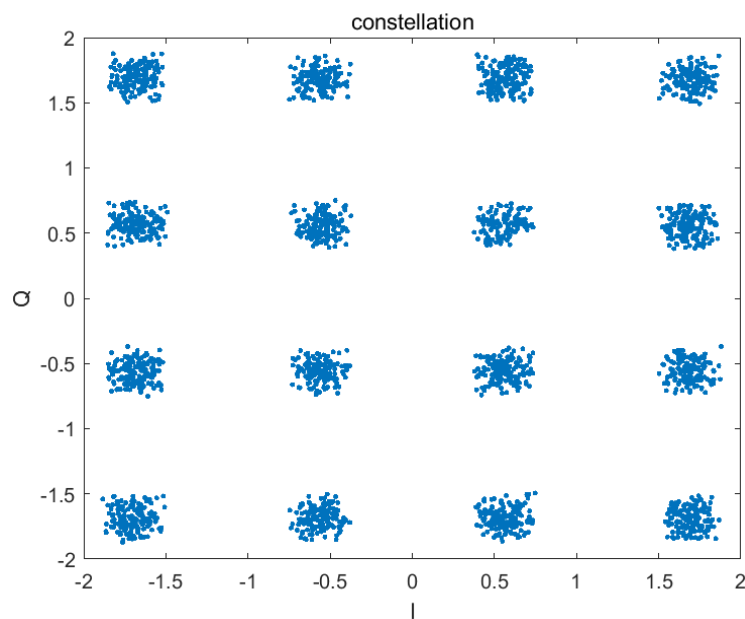


Figure 3.10 Constellation diagram with $c = [1 \ 0 \ .8]$ and $N_{\text{init}} = 1$

From Figure 3.10 it is possible to observe high noise immunity and fewer BERs with forward dispersion delay. And from Figure 3.9 even the open eye diagram can be seen and it can be easily observed that the optimal sampling point changes from 1 to 2 in the case of no dispersion and the following figure shows the corresponding constellation diagram for $N_{\text{init}} = 2$:

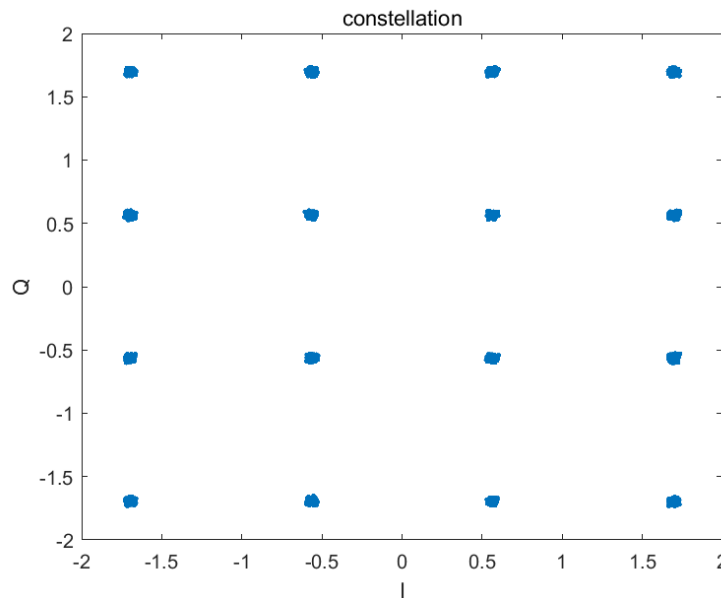


Figure 3.11 Constellation diagram with $c = [1 \ 0 \ .8]$ and $N_{\text{init}} = 2$

From Figure 3.11 it follows that forward dispersion actually causes little or no ISI, just a degree of time delay, whereas the greater the degree of reverse dispersion, the more serious the distortion caused by ISI.

4.1 Problem Restatement

Determine the bit error ratio (BER) (also called the bit error probability) for the noiseless case ($\text{SNR}=1000$ dB), and SNRs of 20 dB (typical SNR of a fixed wireless channel), 12 dB, and 6 dB (typical mobile channel). You will have to alter a constant in the program to get answers that are not 50% BER (i.e. random)! Explain what parameter you had to change and what value you used and then comment on any practical problems you think there are in verifying by simulation the performance of a system with very small BER values (i.e. less than $10e-5$)?

4.2 Content of the answer

The parameter I modified was the delay, which I modified from 10 to 40, and the table below shows the BER corresponding to each SNR in the case where the sampling point is the optimal sampling point.

Table 4.1

SNR(dB)	1000	20	12	6
BER	0	0	0.0264	0.1393

Analogue verification of very small BERs may have the following practical problems for system performance:

1. Long testing time:

Because the BER is very small, a large number of bits need to be transmitted to get the correct statistics. To simulate the verification of a BER of 10^{-5} , at least 10^7 bits would need to be transmitted. (See below for specific proof.) If the transmission rate is 100kbps, the test takes 100 seconds.

Let N be the minimum number of bits to be transmitted, z be the critical value corresponding to a given confidence level in a standard normal distribution table, and c be half the width of the confidence interval. Suppose the confidence level is 95% and the width of the confidence interval is 10 per cent. Then $z = 1.96$, $c = 0.1 \times 10^{-5} = 10^{-6}$. The minimum number of bits to be transmitted is:

$$N = \frac{z^2}{4c^2} = \frac{1.96^2}{4 \times 10^{-12}} = 9.504 \times 10^6 \approx 10^7 (\text{bit})$$

2. The test is not very accurate: because if one is wrong when transmitting 10^5 bits, the BER is 10^{-5} , but this may be a chance event and does not represent the stability of the system. At the same time, if there is no error code during transmission, it may also be a chance event and does not represent the optimality of the system.
3. Untypical test conditions: Since the BER is very small, the test is bound to select matching channel conditions and interference levels, but these may not be typical of the conditions encountered in the actual operation of the communication system, so the test results may not reflect the performance of the system in the real environment.

4.3 Process of solving a problem

The source of the BER error first needs to be determined. Through simulation it was found that the initial sampling point, whatever it was, did not change the probability of the BER approaching 50%. And 50% is the probability of random BERs of 0 and 1. That is to say that an error that occurs during transmission messes up the entire receive sequence. So the process of troubleshooting the exact location of the error began. From back to front, first check the function that converts 16-QAM to a bitstream, change the X_{hat} in the following code to X , namely its original 16-QAM:

```
X_tilde = QAM_Slicer(X_hat,Nbits,'renorm');
X2 = QAM_Demod(X_tilde,Nbits);
```

It can be noticed that B_2 after this decoding is identical to the original bitstream B , so all the later functions from here on are fine. So return to the previous process to check for errors. The previous process goes through three sets of filter functions, and it can be easily found that at vector $c = [1 \ 0 \ 0]$, namely the dispersion-free channel, the signal is unchanged before and after the last set of filters, and therefore the filter there is error-free. Continuing to examine the steps further ahead, the graphs of several key steps are plotted, and the

magnitude of I and Q for each symbol is plotted separately. See the diagrams below for details:

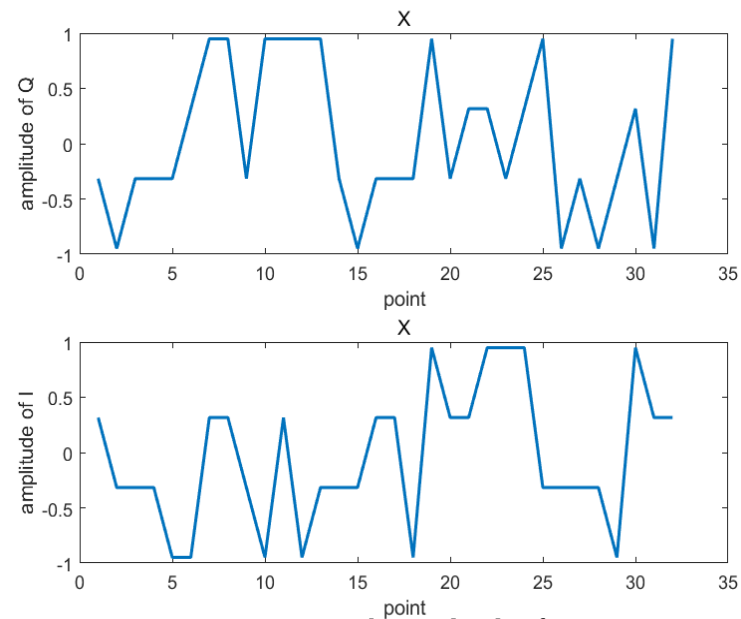


Figure 4.1 the amplitude of X

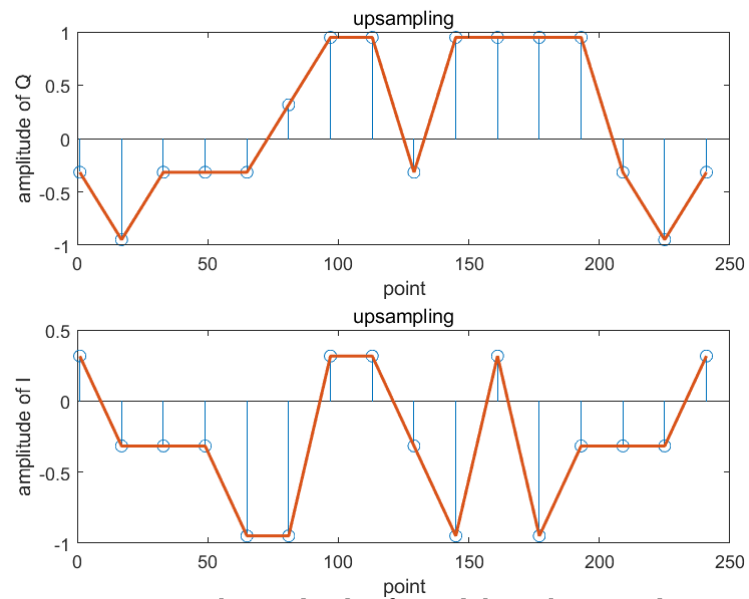


Figure 4.2 the amplitude of signal through upsampling

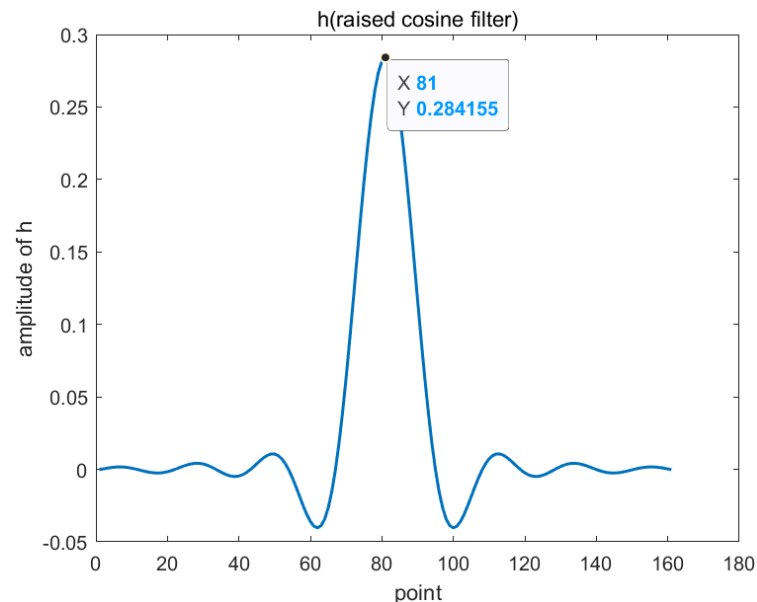


Figure 4.3 the amplitude of h(raised cosine filter)

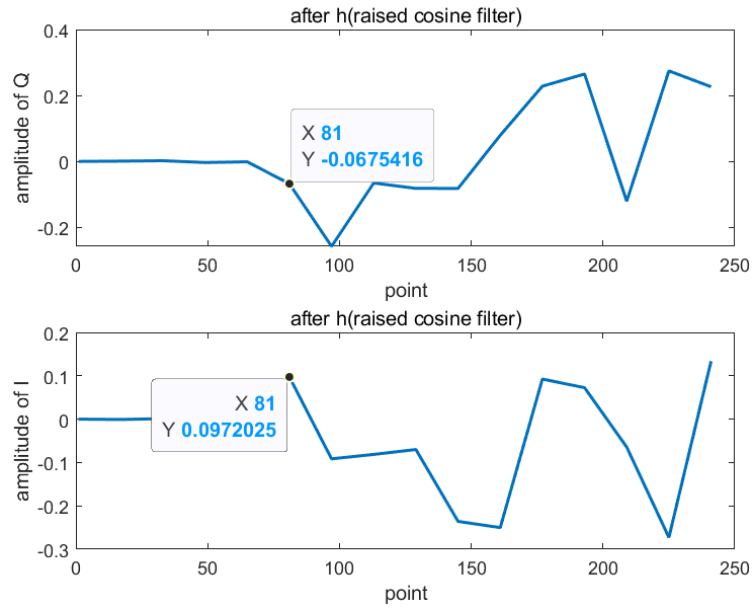


Figure 4.4 the amplitude of signal after h

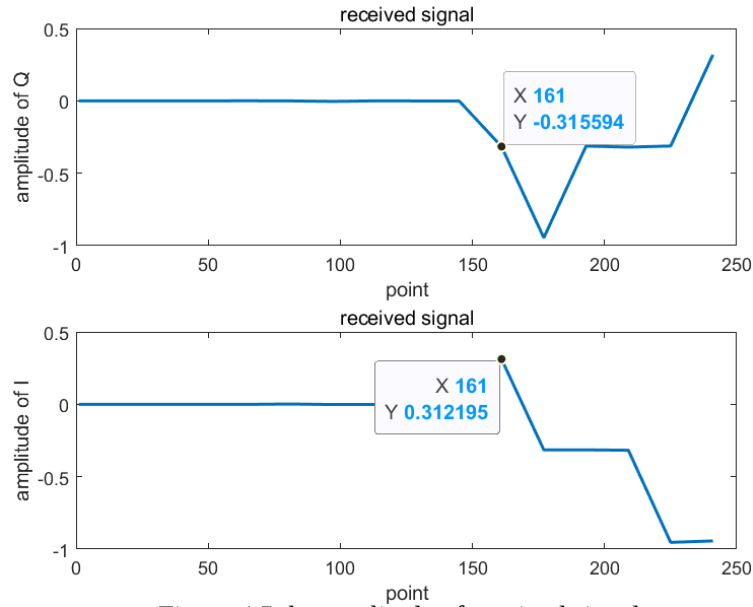


Figure 4.5 the amplitude of received signal

From Figure 4.1 and Figure 4.2 it can be seen that the original signal has been oversampled and no distortion of the image has occurred, therefore the oversampling step is correct. But from figure 4.4 onwards there is a slight distortion of the image, most importantly the delay. It can be noticed from the comparison of Figure 4.4 with Figure 4.2 that Figure 4.4 is delayed by 80 points compared to Figure 4.1, namely point 81 in Figure 4.4 corresponds to point 1 in Figure 4.1, whereas Figure 4.5 shows that the whole sequence is delayed by another 80 points to the right, namely point 161 at the final receiver end corresponds to the first bit of the signal source. And the magnitude map of the final receiver is compared to Figure 4.1. There is no distortion of the magnitude shape except for the misalignment in the time delay, so there is no problem from the sending to the receiving process. It was finally determined that the problem lies in the delay after the final decoding. After traversing a large number of delayB, it was found that the BER was 0 at SNR = 1000dB for delayB = 40.

4.4 Analysis of results

From the comparison between Figure 4.5 and Figure 4.1 it can be seen that the sequence is delayed by a total of 161 during transmission and is delayed to some extent with each pass through the raised cosine filter. From Figure 4.3 it can be seen that h , the highest centre of the raised cosine filter corresponds to a horizontal axis of 81. Therefore it can be seen that the original signal was delayed by 80 when it passed through h . And a total of two passes through h were made, therefore the total delay was 160. When N_{init} is equal to 1, namely the initial sampling point is set to 1, which corresponds to no delay at the receiver side, the total delay is therefore 160. When subsequently decoded into 16-QAM, the 160bit delay is equivalent to 10 16-QAM symbols, which in the end need to be expanded into the original bitstream, so that the 10 symbols delay in symbols now needs to be multiplied by 4 thus giving a bitstream delay of $10 \times 4 = 40$. It is now clear that in the case of an almost noiseless and dispersion-free channel, the delay is generated by the filter. And a closer look at the function that generates the raised cosine filter:

```
h = sqrt(N)*firrcos(10*N,1/N,.5,2,'rolloff','sqrt');
```

It can be found that the first term is of order $10*N$, namely 160. exactly its delay. By changing the first term order to $16*N$, it is indeed found that the time delay becomes, 256, as shown in Figures 4.6, 4.7, and 4.8

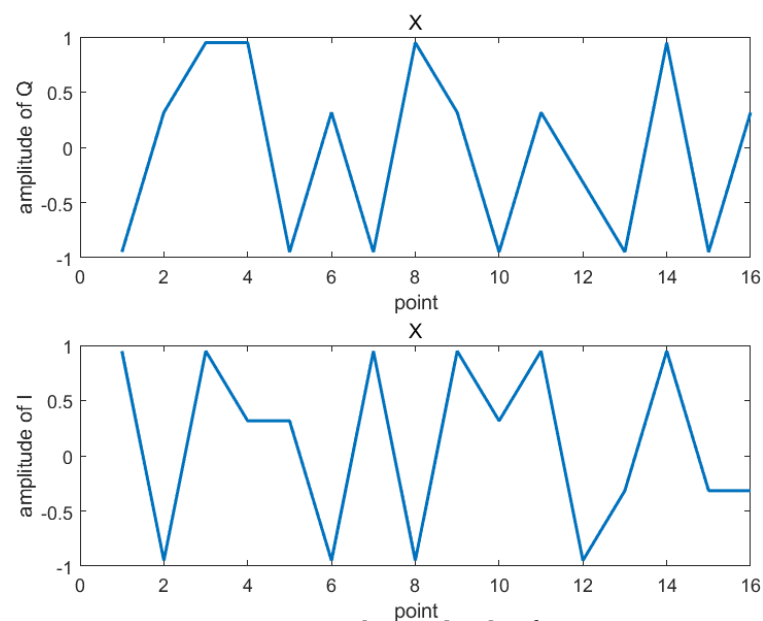


Figure 4.6 the amplitude of X

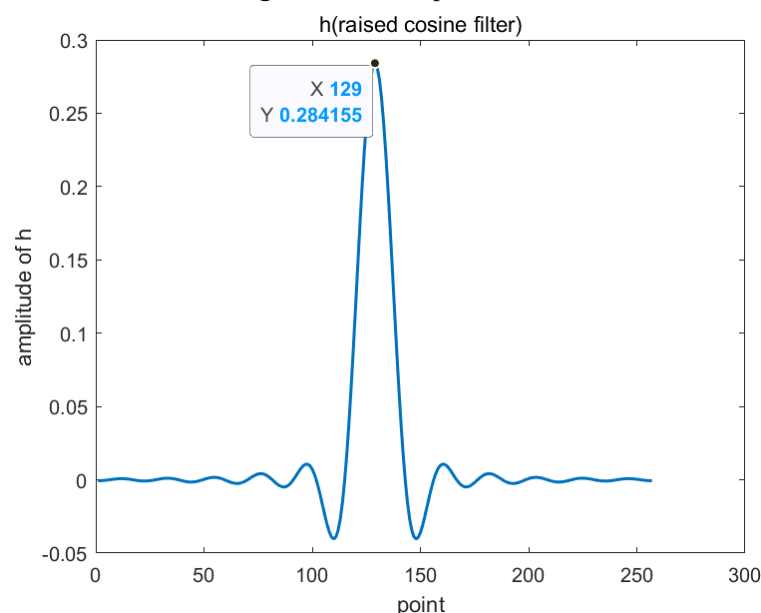


Figure 4.7 the amplitude of h(raised cosine filter)

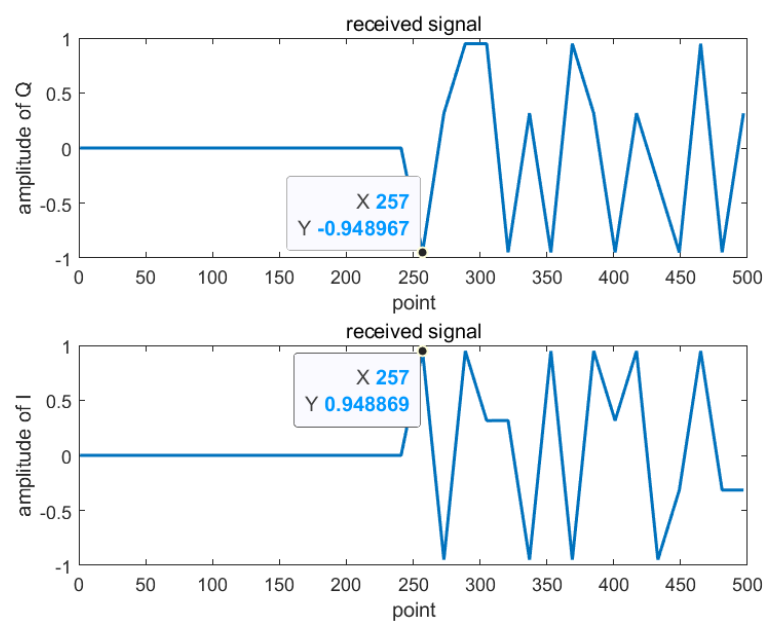


Figure 4.8 the amplitude of received signal

From Figure 4.7 it can be seen that when the order becomes 256, after one h-filter delay of 128, after two is 256, and from the comparison of Figures 4.6 and 4.8 it can be found that the final receiver is indeed delayed by 256. this result is the same as the previous conjecture, and therefore the validation is successful, and the previous analyses about the delay are correct. Therefore to get the correct BER, the relationship between delayB and the delay generated by h is as follows:

Let the filter order be m and the sampling point be Ninit, then

$$delayB = \left\{ \frac{[m - (Ninit - 1)]}{16} \right\} \times 4$$

5.1 Problem Restatement

Add a function to the Matlab code to introduce a simulation of carrier frequency offset of $-1/70$ symbol frequency (i.e. a frequency offset below the simulation carrier frequency of 0Hz). Show a plot of the simulation for 50 symbols only (this is to make it easy to mark) and justify by inspection of the constellation diagram that the offset has been correctly applied.

5.2 Content of the answer

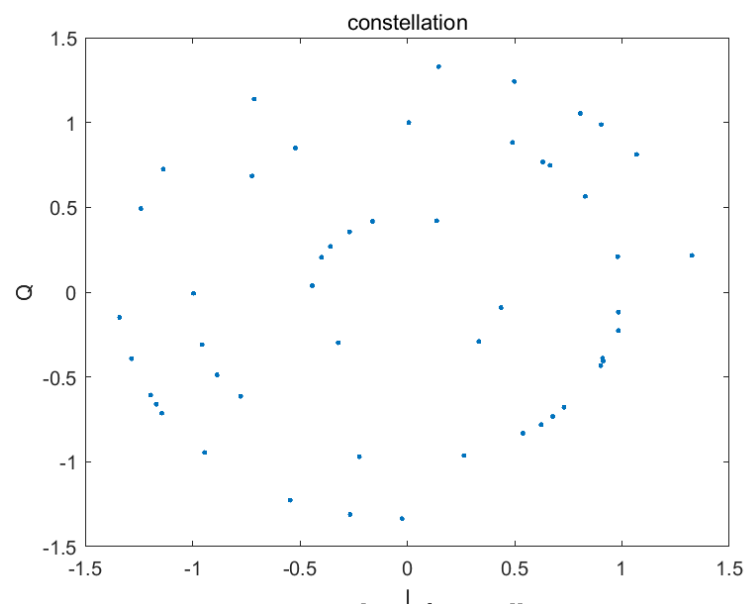


Figure 5.1 50 plots of constellation

It can be observed from Figure 5.1 that the constellation diagram is circular when a frequency offset occurs. This is due to the fact that the phase change caused by the frequency offset creates a periodic change inside 360° . The non-uniformity in the distance between neighbouring points in the constellation diagram is due to the fact that a fixed change in the frequency offset corresponds to a linear increase in the phase deflection with time, and therefore the phase change between neighbouring points is not the same with respect to the original point position. And since the frequency offset is $-1/70\text{Hz}$ per symbol, each symbol corresponds to a counterclockwise rotation of the original symbol, hence the counterclockwise spiral in Figure 5.1. So the offset has been correctly applied.

5.3 Process of solving a problem

See the code added with the receive filtering for details.

```
s2 = filter(h,1,s_hat);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
fshift = -1/70;%frequency shift
k = 0;
a = 1;
p = ones(1,40000);
for i = 1:40000
    k = k+1;
    if k<= 16
        p(1,i) = fshift*a*p(1,i)*2*pi;
    else
        k = 1;
        a = a+1;
        p(1,i) = fshift*a*p(1,i)*2*pi;
    end
end
p2 = angle(s2)+p;
p3 = abs(s2);
s2 = p3.*exp(1i*p2);
```

Replace the part about drawing a constellation diagram with:

```
Ninit = 1; % determine sampling point (0<Ninit<=N)
X_hat = s2(Ninit:N:end); % "sample" the signal
% plot received constellation
figure(2); clf;
set(gcf,'Color',[1 1 1]);
plot(X_hat(11:60),'.');
title('constellation'); xlabel('I'); ylabel('Q');
```

Due to the delay of the raised cosine filter, the correct constellation diagram can only be seen starting at 11 points.

5.4 Analysis of results

If you complete the constellation diagram, see below:

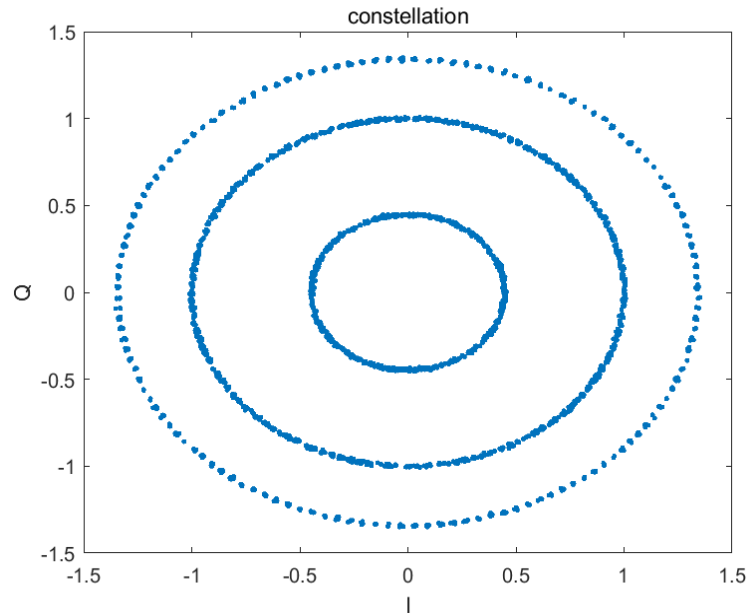


Figure 5.2 Constellation

Three very clear circles can be observed from Figure 5.2, the reason for this phenomenon is firstly because the phase deviation caused by the frequency offset corresponds to the 360° periodicity, and secondly because the 16-QAM corresponds to the constellation diagram with the origin to the centre of each point has three radii. See the red line in Figure 5.3.

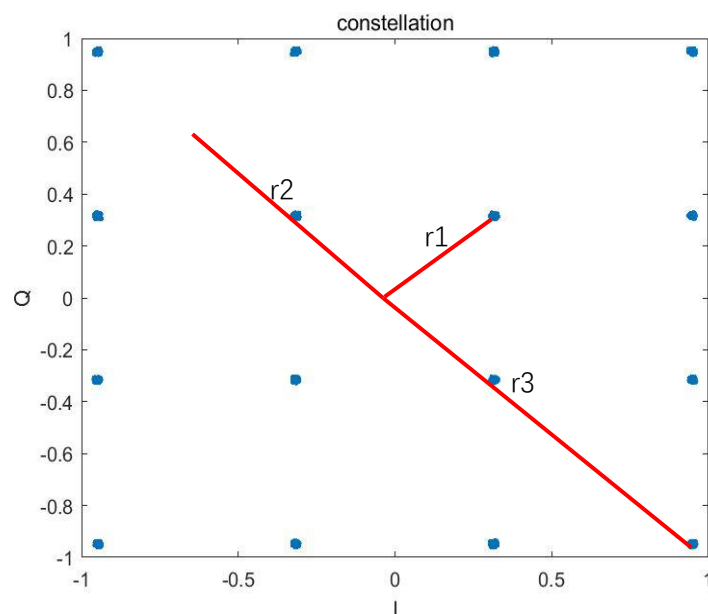


Figure 5.3 Constellation diagram with an initial sampling point of 1

Since each symbol has a frequency offset of $-1/70\text{Hz}$, each symbol has a frequency offset of $-$ about $1/70 \times 2 \times \pi = -0.089$ radians, which corresponds to an angle of about -5.14° . And due to the time delay generated by the raised cosine filter, the first 16-QAM symbol on the

transmitter side is actually equal to the 11th 16-QAM symbol on the receiver side, so it corresponds to a counterclockwise deflection of the first symbol of about $0.089 \times 11 = 0.979$ radians. See the figure below:

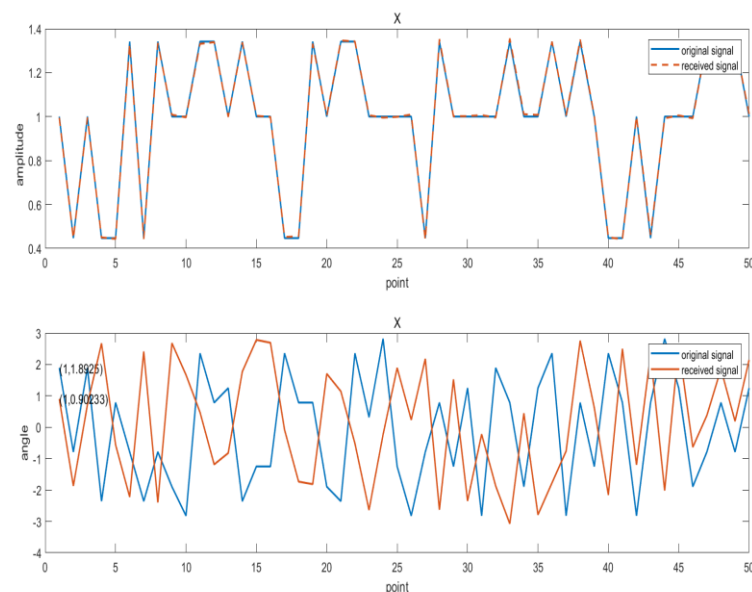


Figure 5.4 Comparison of frequency offset before and after

From the angular plots before and after the frequency offset in Figure 5.4, it can be clearly observed that the radian shift in the start position is as predicted. And since it is the frequency offset of the symbols, there is no change in the magnitude.

6.1 Problem Restatement

Add a function to the Matlab code to estimate the value of Ninit (the optimum sampling point). The function may use many data symbols to form the estimate. Do not use any information such as timing or the noiseless signal from the transmitter part of the simulation, it is not available to a distant receiver. Show your results as a set of three histograms (one for each value of SNR) showing the distribution of Ninit for each SNR value. Note that Ninit is expected to be correct for all values of SNR above 10 dB, so concentrate your results in the region of SNR below 10 dB.

6.2 Content of the answer

The following graphs are all based on the statistical results of simulating 100 communication transmissions.

Method 1: Traversal

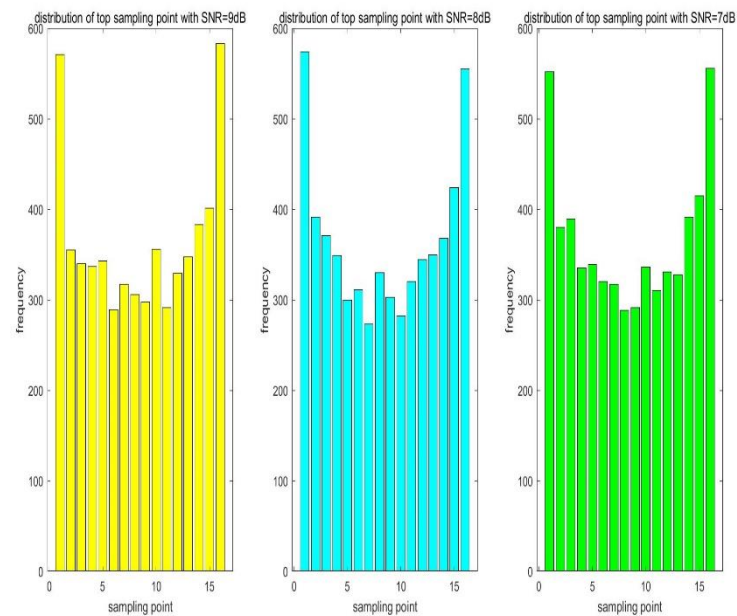


Figure 6.1 First set of histograms

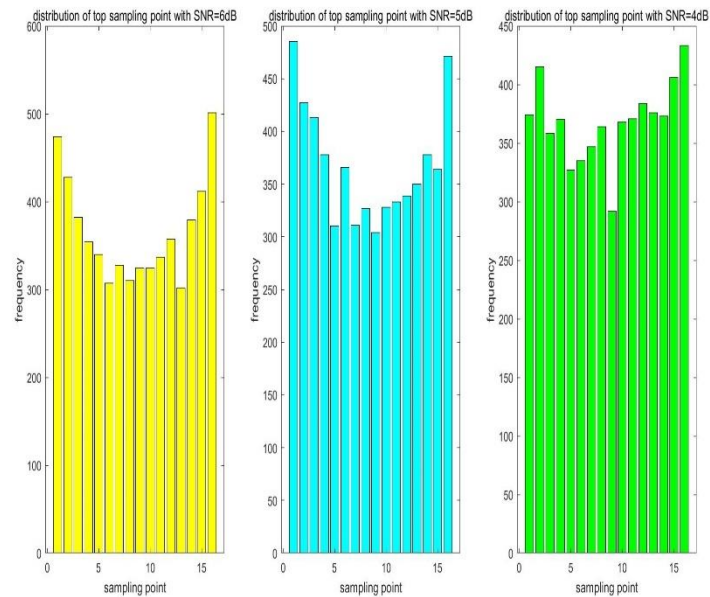


Figure 6.2 Second set of histograms

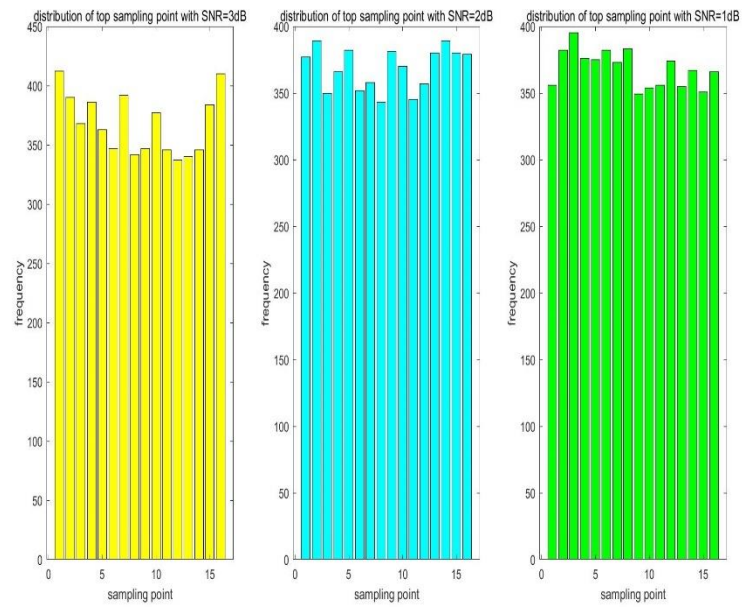


Figure 6.3 Third set of histograms

Method 2: Filtered Delay Estimation

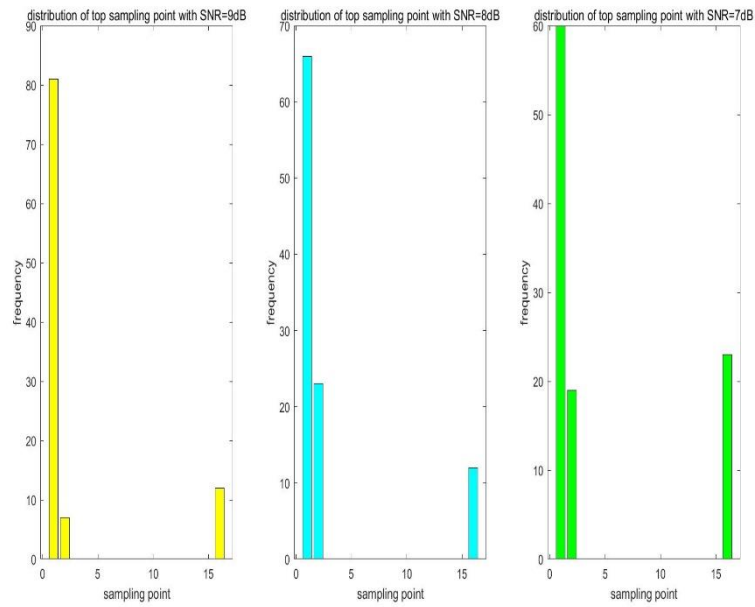


Figure 6.4 First set of histograms

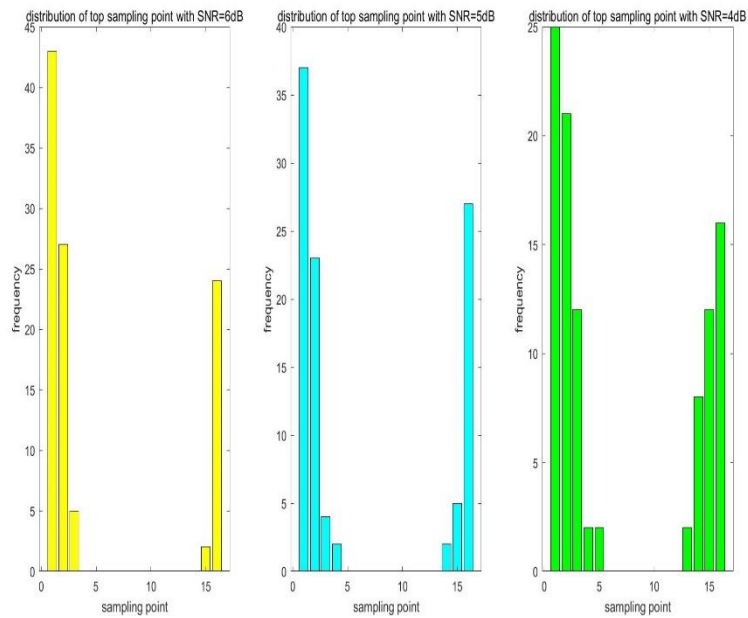


Figure 6.5 Second set of histograms

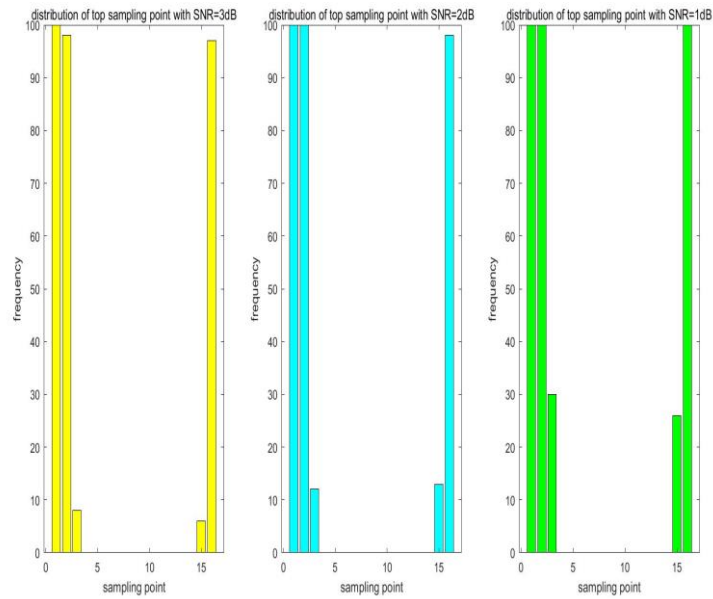


Figure 6.6 Third set of histograms

The functions and main programs corresponding to the two methods are shown in the appendix.

6.3 Process of solving a problem

The value that characterises the optimal sampling point is first determined. This is because the method of determining the best sampling point based on the eye diagram fails when there is too much noise. Therefore the most reliable approach namely BER is used for characterisation. A small BER means that the communication system is highly reliable and corresponds to a good sampling point. However, there are many reasons for the formation of BER, and since this question focuses on noise, the effect of dispersive channels is not considered. And considering that the actual long range channel does not allow the use of the original code without noise, it is necessary to add a convolutional code at the transmitter side to recover the original signal at the receiver side. Feedback checking could have been considered, but since the channel considered in this problem is extremely noisy, the received code sent back to the sender will also be disturbed, so feedback checking to calculate the BER is not used to evaluate the distribution of the optimal sampling points. The more commonly used convolutional code configuration of CC (2, 1, 7) is chosen. And use the Viterbi solving method to recover the original code from the transmitter. The algorithm of BER is to calculate the Hamming distance between the code received at the receiving, and its original code reduced from the convolutional code received at the receiving, and then divide it by the code length. The occurrence of the best sampling point is obviously a probabilistic event when there is too much noise, and the delays in real communication systems are not always perfectly matched. Therefore, the delay (delayB) is first set to range from 1 to 60. The reason for this range is that, according to the formula derived at the end of 4.4 for delay compensation versus sampling point, the optimal delay compensation is at largest 40 for a range of sampling points from 1 to 16. The range of

delays must therefore include it. In the function written for the best sampling point, all the sampling points are traversed, and each sampling point has to participate in the calculation of BER in the delay compensation from 1 to 60, and then take the minimum BER of each different delay, and then use the matrix to record the number of times that the corresponding sampling point corresponds to the process of becoming the best sampling point, and then finally in the main program, the process is traversed 100 times to get the relatively objective statistical data. Finally, in the main program, the process is 100 times to get relatively objective statistics. This method has the advantage of introducing randomness in the delay. But the disadvantage is to introduce the time delay into the traversal range, the arithmetic is large.

The second method is based on the formula of 4.4 for the relationship between the time delay and the best sampling point. Although the time delay has randomness in the actual communication system, its randomness is also consistent with the relationship between the time delay and the optimal sampling point, so the formula of 4.4 can be used to round the number of 16-QAM symbols of the desired delay, and then get the optimal delay bit number corresponding to each sampling point, and finally the distribution of the optimal sampling point can be obtained by traversing the range for 100 times under different noises. The advantage of this method is to greatly simplify the amount of arithmetic, the disadvantage is that in the process of finding the optimal delay, some of the optimal delay is not an integer multiple of the 16-QAM symbols of the sampling points in the rounding process there will be a certain error, which will affect the statistical distribution. So ultimately a combination of the two methods is more robust.

Both methods require decoding of the received convolutional code at the receiver side to restore it to the original code.

6.4 Analysis of results

It can be noticed from most of the above graphs that the sampling point that becomes the best sampling point the most times is sampling point 1, then sampling point 16 and point 2. And as the sampling points get further and further away from sampling points 1 and 16, namely towards the middle of the data, the less likely it is that these sampling points will become the best sampling points. The essential reason for these statistical phenomena lies in the equation for delay versus sampling point pushed at the end of 4.4, the filter order be m and the sampling point be N_{init} :

$$delayB = \left\{ \frac{[m - (N_{init} - 1)]}{16} \right\} \times 4$$

Delay compensation for integer 16-QAM is available only when the sampling point is 1, namely $160 \div 16 = 10$.

and the next point that satisfies this is 17, but it is not considered as the best sampling point in this question, so point 16 and point 2 become the best sampling point with the second highest number of times. It can also be observed in the above figure that the distribution of the best sampling points has symmetry. This is due to the fact that the optimal delay compensation is expressed in 16-QAM notation when the sampling point is 9:

$$[160 - (9 - 1)] \div 16 = 9.5.$$

This means that starting from sampling point 9, the optimal delay compensation of the sampling point smaller than it tends to be 40 and the optimal delay compensation of the sampling point larger than it tends to be 36, and thus there are two different optimal delay compensations on either side of it, which leads to a symmetrical distribution of optimal sampling points. However, it can also be found that not every picture becomes the best sampling point number of 1. For example, in Figure 6.3, it is almost impossible to distinguish which sampling point has been the best sampling point with the biggest times. This is because as the noise continues to expand, the role of the time delay matching the best sampling point becomes smaller and smaller, and the statistical results become closer and closer to the statistical characteristics of the noise. Another reason is because the convolutional code is also affected by noise during transmission, and when the noise is as large as in Figure 6.3, the error of the original code reduced by the convolutional code increases, thus further increasing the difficulty of determining the optimal distribution of sampling points with BER. Looking at the distribution of the best sampling points for signal-to-noise ratios from 9dB to 1dB, which can be seen that as the signal-to-noise ratio continues to decrease, the number of best sampling points at the ends gradually decreases, and those in the middle gradually increase. The most significant difference in the distribution of optimal sampling points estimated with filtered delay estimation from Figures 6.1 to 6.3 is that a large number of sampling points in the middle do not have a chance to be the optimal sampling points. This is because this approach circumvents the possibility of other points being the best sampling point due to noise. Looking closely at Figures 6.1 to 6.3, subtracting the number of times the best sampling point at each side becomes the best sampling point from the number of times the sampling point in the center becomes the best sampling point, and the approximate distribution matches Figures 6.4 to 6.6. namely the extra number of times the sampling points at each side become the best sampling point comes from the exact matching of the time delays.

Summaries:

There were a total of six questions in this coursework, and my takeaways from completing these tasks were:

1. Learnt to look at the advantages and disadvantages of communication systems from eye diagrams and constellation diagrams, optimal sampling point locations.
2. Gained a good understanding of the effects of noise on communication systems.
3. Gain insight into the effect of dispersion channels on communication transmissions.
4. Gained a good understanding of the delay caused by filters in communication systems.
5. Found the relationship between the order of the filter and the sampling point.
6. Explored in depth the effect of frequency offset on the received signal.
7. Mastered some methods of finding the optimum sampling point.
8. Gained insight into BER.
9. Increased understanding of the overall digital communications system.
10. Learning to use convolutional codes to recover the original code at the receiving using Viterbi solving method.

Appendix

Code for Figures 4.1 to 4.5:

DigiTrans is unchanged except for the change from $N_{init} = 10$ to $N_{init} = 1$. The attached programme is shown below:

```
obn = 16;
figure(1)
subplot(2,1,1)
set(gcf,'Color',[1 1 1]);
plot(real(X(1:32)), 'LineWidth',1.5);
xlabel("point");
ylabel("amplitude of Q");
title("X");
hold on
subplot(2,1,2)
plot(imag(X(1:32)), 'LineWidth',1.5);
set(gcf,'Color',[1 1 1]);
xlabel("point");
ylabel("amplitude of I");
title("X");
figure(2)
y1r = real(Xup(1:16:(obn-1)*16+1));
y2r = real(Xup(1,1:16:(obn-1)*16+1));
subplot(2,1,1);
set(gcf,'Color',[1 1 1]);
stem(1:16:(obn-1)*16+1,y1r)
hold on
plot(1:16:(obn-1)*16+1, y2r, 'LineWidth',1.5)
xlabel("point");
ylabel("amplitude of Q");
title("upsampling");
hold off
y1i = imag(Xup(1:16:(obn-1)*16+1));
y2i = imag(Xup(1,1:16:(obn-1)*16+1));
subplot(2,1,2);
set(gcf,'Color',[1 1 1]);
stem(1:16:(obn-1)*16+1,y1i)
hold on
plot(1:16:(obn-1)*16+1, y2i, 'LineWidth',1.5)
```

```

xlabel("point");
ylabel("amplitude of I");
title("upsampling");
hold off
figure(3)
sr = real(s(1:16:(obn-1)*16+1));
subplot(2,1,1)
set(gcf,'Color',[1 1 1]);
plot(1:16:(obn-1)*16+1, sr,'LineWidth',1.5)
xlabel("point");
ylabel("amplitude of Q");
title("after h(raised cosine filter)");
subplot(2,1,2)
set(gcf,'Color',[1 1 1]);
si = imag(s(1:16:(obn-1)*16+1));
plot(1:16:(obn-1)*16+1, si,'LineWidth',1.5)
xlabel("point");
ylabel("amplitude of I");
title("after h(raised cosine filter)");
figure(4)
set(gcf,'Color',[1 1 1]);
plot(h,'LineWidth',1.5)
xlabel("point");
ylabel("amplitude of h");
title("h(raised cosine filter)");
figure(5)
s2r = real(s2(1:16:(2*obn-1)*16+1));
subplot(2,1,1)
set(gcf,'Color',[1 1 1]);
plot(1:16:(2*obn-1)*16+1, s2r,'LineWidth',1.5)
xlabel("point");
ylabel("amplitude of Q");
title("received signal");
subplot(2,1,2)
set(gcf,'Color',[1 1 1]);
s2i = imag(s2(1:16:(2*obn-1)*16+1));
plot(1:16:(2*obn-1)*16+1, s2i,'LineWidth',1.5)
xlabel("point");
ylabel("amplitude of I");
title("received signal");

```

Code for figure 5.4:

```
subplot(2,1,1)
set(gcf,'Color',[1 1 1]);
plot(abs(X(1:50)),'LineWidth',1.5);
xlabel("point");
ylabel("amplitude");
title("X");
hold on
plot(abs(X_hat(11:60)),'--','LineWidth',1.5);
legend('original signal','received signal');
hold off

subplot(2,1,2)
plot(angle(X(1:50)),'LineWidth',1.5);
set(gcf,'Color',[1 1 1]);
xlabel("point");
ylabel("angle");
title("X");
b1 = angle(X(1:50));
y1 = 1:50;
str = ['(', num2str(y1(1,1)), ',', num2str(b1(1,1)), ')'];
text(y1(1,1), b1(1,1),str);
hold on
plot(angle(X_hat(11:60)),'LineWidth',1.5);
legend('original signal','received signal');
b2 = angle(X_hat(11:60));
str = ['(', num2str(y1(1,1)), ',', num2str(b2(1,1)), ')'];
text(y1(1,1), b2(1,1),str);
hold off
```

Code for figure 6.1:(6.1 to 6.3 are vary similar)

main program:

```
clear all
L = 100;
N = 16;
y1 = zeros(L,N);
y2 = zeros(L,N);
y3 = zeros(L,N);
```

```
for g = 1:L
    st = optimalsp(9);
    st1 = optimalsp(8);
    st2 = optimalsp(7);
    y1(g,:) = st;
    y2(g,:) = st1;
    y3(g,:) = st2;
end

hy1=sum(y1);
hy2=sum(y2);
hy3=sum(y3);
subplot(1,3,1)
set(gcf,'Color',[1 1 1]);
bar(hy1,'facecolor','yellow');
xlabel("sampling point")
ylabel("frequency")
title("distribution of top sampling point with SNR=9dB")
subplot(1,3,2)
set(gcf,'Color',[1 1 1]);
bar(hy2,'facecolor','cyan');
xlabel("sampling point")
ylabel("frequency")
title("distribution of top sampling point with SNR=8dB")
subplot(1,3,3)
set(gcf,'Color',[1 1 1]);
bar(hy3,'facecolor','green');
xlabel("sampling point")
ylabel("frequency")
title("distribution of top sampling point with SNR=7dB")
```

optimalsp1 function:

[illegible]

[illegible]

```

c = [1 0 0]; % CIR (at N x symbol rate)
s_hat1 = filter(c,1,s);%2
sigma_x = std(s_hat1);
Ls = length(s_hat1);
noise = (randn(1,Ls) + 1j*randn(1,Ls))*sqrt(N)/sqrt(2);
s_hat = s_hat1 + sigma_x*10^(-SNR/20)*noise;%c(t)
sigPower = sum(abs(encoded_od(:)).^2)/length(encoded_od(:));
reqSNR = 10^(SNR/10);
noisePower = sigPower/reqSNR;
noise = sqrt(noisePower)*randn(size(encoded_od));
noisy_od = encoded_od + noise;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% receive filtering
s2 = filter(h,1,s_hat);%filter f(t)->r(t)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% sampling at symbol rate

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Ninit = i; % determine sampling point (0<Ninit<=N)
% X_hat = s2(129:16:end); % "sample" the signal
X_hat = s2(i:16:end);
X_tilde = QAM_Slicer(X_hat,Nbits,'renorm');
X2 = QAM_Demod(X_tilde,Nbits);
% Hard judgement demodulation
hard_decisions = noisy_od > 0.5;
% decode
decoded = vitdec(hard_decisions, trellis, m, 'trunc', 'hard');
%B2 = SymbolToBitStream(X2,Nbits);
Lx = length(X2);
Lb = Lx*Nbits;
B2 = zeros(1,Lb);
for n = Nbits:-1:1
    U = floor(X2./2^(n-1));
    B2(Nbits-n+1:Nbits:Lb)=U;
    X2 = X2 - 2^(n-1)*U;
end
for delayB = 1:60
    diff = decoded(1:end-delayB) - B2(delayB+1:end);
    BER = sum(abs(diff))/(length(decoded)-delayB);

```

```

de(delayB,i) = BER;
end
end
y1 = zeros(60,16);
for i = 1:16
for f = 1:60
[m1,n1] = find(de==min(de(f,:)));
if n1 == i
y1(f,i) = 1;
end
end
end
y = sum(y1);

```

Code for figure 6.4:(6.4 to 6.6 are vary similar)

main program:

```

L = 100;
y1 = zeros(16,L);
y2 = zeros(16,L);
y3 = zeros(16,L);
for g = 1:L
for v = 1:16
st = optimalsp(9,16);
lp = length(st);
if lp==1
y1(st,g)=1;
else
for b = 1:lp
y1(st(1,b),g) = 1;
end
end
end
for v = 1:16
st = optimalsp(8,16);
lp = length(st);
if lp==1
y2(st,g)=1;
else

```

```

        for b = 1:lp
            y2(st(1,b),g) = 1;
        end
    end
end
for v = 1:16
    st = optimalsp(7,16);
    lp = length(st);
    if lp==1
        y3(st,g)=1;
    else
        for b = 1:lp
            y3(st(1,b),g) = 1;
        end
    end
end
end
hy1=sum(y1,2)';
hy2=sum(y2,2)';
hy3=sum(y3,2)';
subplot(1,3,1)
set(gcf,'Color',[1 1 1]);
bar(hy1,'facecolor','yellow');
xlabel("sampling point")
ylabel("frequency")
title("distribution of top sampling point with SNR=9dB")
subplot(1,3,2)
set(gcf,'Color',[1 1 1]);
bar(hy2,'facecolor','cyan');
xlabel("sampling point")
ylabel("frequency")
title("distribution of top sampling point with SNR=8dB")
subplot(1,3,3)
set(gcf,'Color',[1 1 1]);
bar(hy3,'facecolor','green');
xlabel("sampling point")
ylabel("frequency")
title("distribution of top sampling point with SNR=7dB")

```

optimalsp function:

```

function y = optimalsp(snr)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% additive white Gaussian noise (AWGN)

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

N = 16;

SNR = snr;

LB = 10000;          % number of bits

B1 = rand(1, LB);

B = round(B1);

de = zeros(1, N);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% conversion to 16-QAM symbol

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

Nbits = 2;

% group Nbits successive bits into each I and Q

Lb = length(B);

if mod(Lb, Nbits) ~= 0
    error('Bitstream length is incompatible with symbol length');
end

Lx = Lb/Nbits;

x = zeros(1, Lx);

for n = 1:Nbits
    x = x + 2^(Nbits-n)*B(n:Nbits:Lb);
end

% implement 2^(2*Nbits)-QAM s(n)

X = QAM_Mod(x, Nbits);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

od = B; % original data

% Convolutional code parameters

n = 2; % number of output bits

k = 1; % number of input bits

m = 7; % Constraint length

% define polynomial

g = [171 133];

% produce trellis structure

```

```

trellis = poly2trellis(m, g);
% convolutional coding of data
encoded_od = convenc(od, trellis);

for ij = 1:N
Xup = zeros(1,length(X)*N);
Xup(1:N:end) = X;
h = sqrt(N)*firrcos(10*N,1/N,.5,2,'rolloff','sqrt');
s = filter(h,1,Xup);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% filtering with channel impulse response
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
c = [1 0 0]; % CIR (at N x symbol rate)
s_hat1 = filter(c,1,s);%2
sigma_x = std(s_hat1);
Ls = length(s_hat1);
noise = (randn(1,Ls) + 1j*randn(1,Ls))*sqrt(N)/sqrt(2);
s_hat = s_hat1 + sigma_x*10^(-SNR/20)*noise;%c(t)
sigPower = sum(abs(encoded_od(:)).^2)/length(encoded_od(:));
reqSNR = 10^(SNR/10);
noisePower = sigPower/reqSNR;
noise = sqrt(noisePower)*randn(size(encoded_od));
noisy_od = encoded_od + noise;
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% receive filtering
s2 = filter(h,1,s_hat);%filter f(t)->r(t)
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% sampling at symbol rate
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
Ninit = ij; % determine sampling point (0<Ninit<=N)
X_hat = s2(ij:16:end);
X_tilde = QAM_Slicer(X_hat,Nbits,'renorm');
X2 = QAM_Demod(X_tilde,Nbits);
% Hard judgement demodulation
hard_decisions = noisy_od > 0.5;
% decode

```

```

decoded = vitdec(hard_decisions, trellis, m, 'trunc', 'hard');
%B2 = SymbolToBitStream(X2,Nbits);
Lx = length(X2);
Lb = Lx*Nbits;
B2 = zeros(1,Lb);
for n = Nbits:-1:1
    U = floor(X2./2^(n-1));
    B2(Nbits-n+1:Nbits:Lb)=U;
    X2 = X2 - 2^(n-1)*U;
end
if ij == 9
    delayB = 9*4;
else
    delayB = round((10*N+1-ij)/16)*4;% to compensate delays in channel & TX/RX
end
diff = decoded(1:end-delayB) - B2(delayB+1:end);
BER = sum(abs(diff))/(length(decoded)-delayB);
de(ij) = BER;
end
[m1,n1] = find(de==min(de));
y = n1;

```